

面向 AI 芯片的 Qwen3-VL-2B 高效推理优化技术报告（复赛）

摘要

本工作面向 Qwen3-VL-2B-Instruct 多模态大模型，在复赛官方评测平台（阿里云 PPU 服务器）上对视觉问答（VQA）推理进行端到端优化。全部优化按"系统层"与"算法层"两条主线组织，统一目标为：在保持准确率基本不变的前提下，降低首 Token 时延（TTFT）、提升生成吞吐量（Throughput）。

- 初赛（系统层）：在不改动算法、不降低精度、不降低分辨率的前提下，仅以系统工程手段（优化注意力计算、缓存复用与 GPU 调度方式），将 TTFT 由约 80 ms 降至约 20 ms（下降约 75%），生成吞吐由约 55 tok/s 提升至约 299 tok/s（约 5.4 倍），准确率基本无损。
- 复赛（算法层）：在系统层基础上引入 INT8 量化、推测解码、自适应分辨率等算法优化，进一步提升吞吐。最终提交版本官方吞吐达约 5634 tok/s（约为基线的 100 倍），TTFT 维持在约 20 ms。

一、研究背景与目标

1.1 应用场景与赛题

视觉语言模型（VLM）在视觉问答、图像理解、文档与票据识别、多模态交互等场景具有广泛的应用价值。Qwen3-VL-2B-Instruct 是通义千问系列的 20 亿参数多模态模型，由视觉编码器（ViT，24 层）与语言解码器（Transformer Decoder，28 层）两部分构成，支持动态分辨率输入，并通过 Patch Merger（图块合并器，将相邻 2×2 的视觉块合并为一个）与 DeepStack（深层特征注入，将视觉编码器的中间层特征注入语言模型）完成图文融合。

赛题要求：在仅修改推理包装代码、不改动模型本身的前提下，于保持 VQA 准确率的同时，最小化首 Token 时延（TTFT）、最大化生成吞吐量（Throughput）。主要约束包括：批大小固定为 1；禁止硬编码答案、修改数据集、调用外部服务、替换或重新训练模型、直接改写权重数值，以及针对评测样本的过拟合。

1.2 评测指标

$$\text{得分} = 0.4 \times \text{准确率} + 0.3 \times \text{TTFT 提升率} + 0.3 \times \text{吞吐提升率}$$

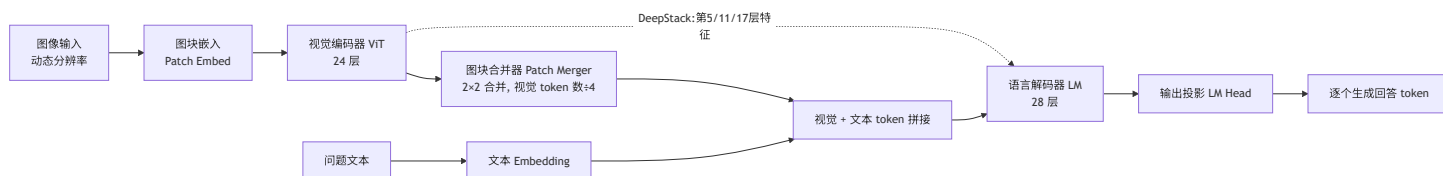
- **TTFT (首 Token 时延)**: 自输入准备至生成第一个 token (token 为模型处理文本的基本单位, 约相当于一个词或子词) 的端到端时间, 涵盖图像编码、文本编码、图文融合, 以及对完整输入提示的一次性处理 (即 prefill 阶段)。官方基线约 80 ms。
- **吞吐量 (Throughput)**: 端到端的 token 生成速率 (tokens/秒)。官方基线约 55 tok/s。
- **准确率 (Accuracy)**: 验证集上的 VQA 软匹配准确率 (支持多个标准答案)。基线约 0.7。

评分公式是本职工作各项取舍的依据: 准确率占 0.4, TTFT 与吞吐各占 0.3。

1.3 硬件与软件平台

复赛官方评测在阿里云 PPU 服务器上进行。PPU 是面向 AI 推理的加速器, 编程接口与 CUDA 生态兼容, 故本项目以 CUDA、Triton、Flash Attention 等技术实现的自定义内核 (内核指在加速器上执行的底层计算程序) 可直接运行。软件环境为 CUDA 12.8、PyTorch 2.8、Triton 3.4、Flash Attention 2.x, 并与官方临时容器对齐至 transformers 4.57。

1.4 模型架构



整体数据流为: 图像经视觉编码器与 Patch Merger 得到视觉 token, 与问题文本 token 拼接后送入语言解码器, 逐个生成回答 token。其中 DeepStack 机制取视觉编码器第 5、11、17 层的中间特征, 分别注入语言解码器最前面的三层, 为语言模型补充不同层次的视觉细节。模型总参数约 20 亿; 由于每生成一个 token 都要把约 3.49 GB 的全部权重从显存读取一遍, 而实际计算量却很小, 生成速度主要受限于显存的数据搬运速度而非算力, 即属于典型的显存带宽受限任务。

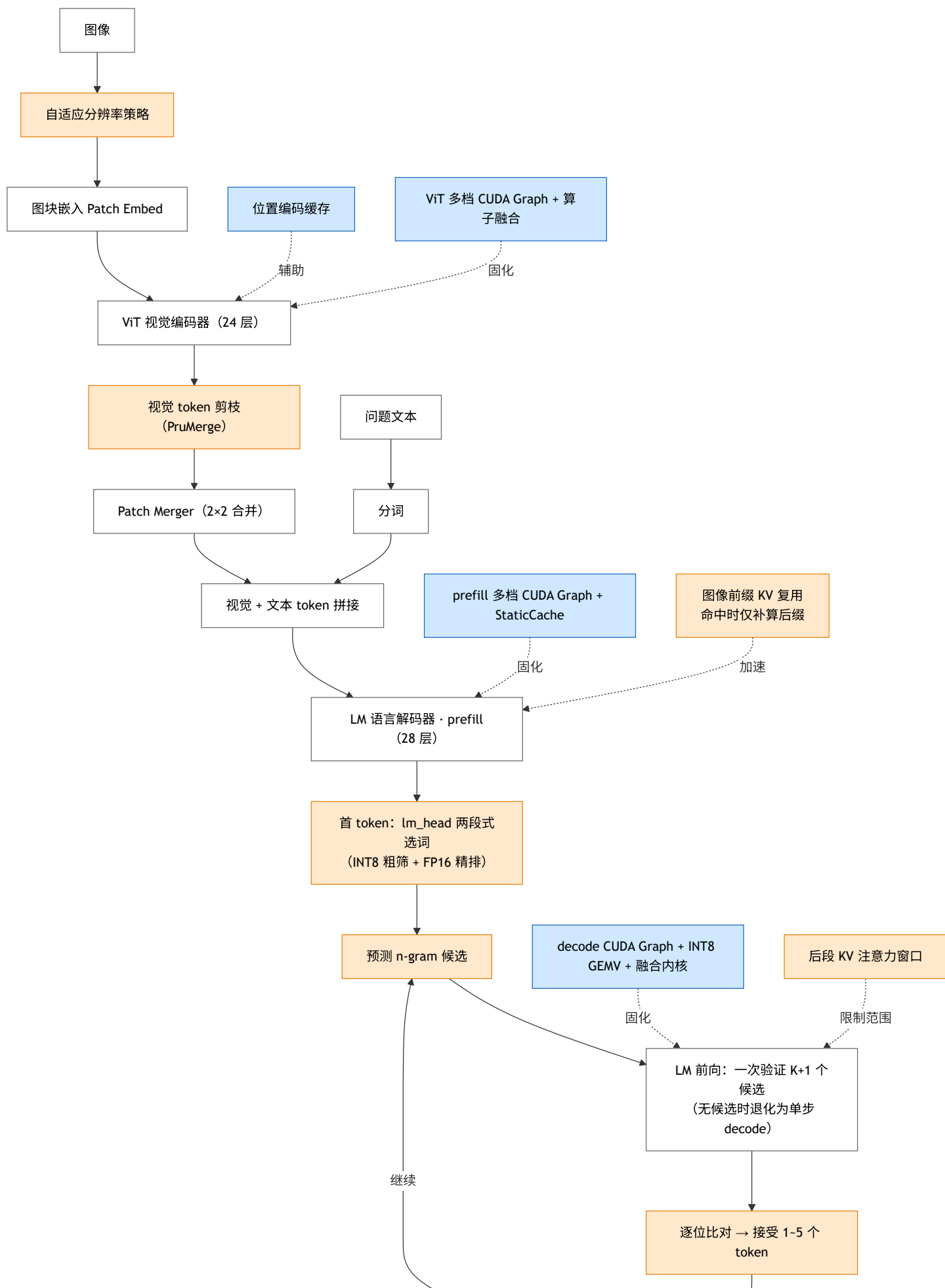
二、相关工作

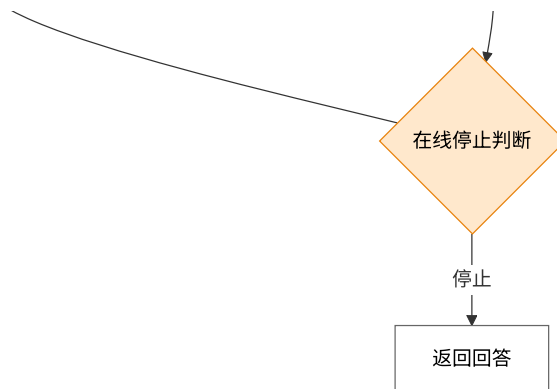
- **Flash Attention**: 通过分块计算, 避免一次性构造庞大的中间注意力矩阵, 使显存访问量随序列变长时的增长由平方级降为线性级, 是当前注意力计算的工业标准。
- **CUDA Graph**: 将一连串 GPU 计算预先录制为一张静态图, 执行时一次性提交, 省去逐个算子从 Python 端发起的调度开销。
- **算子融合 (Kernel Fusion)**: 将相邻的小算子合并为一个, 减少中间结果在显存中的反复读写。

- **权重量化 (Weight-only INT8)**：以 INT8 (8 位整数) 存储权重、计算时仍用 FP16 (16 位半精度浮点)，针对带宽受限的 decode 阶段减少每步需读取的权重字节数。
- **推测解码 (Speculative Decoding / n-gram lookahead)**：以低成本方式（依据文本中连续若干词的出现统计，即 n-gram）先预测若干后续 token，再经一次前向集中验证、一次接受多个，从而打破自回归逐 token 生成的串行瓶颈。
- **vLLM**：以 PagedAttention、连续批处理等技术为代表的成熟推理引擎，主要面向多请求高并发场景；本工作面向批大小固定为 1 的单请求场景，其多数面向并发的优化并不适用，故未直接采用，而是自行实现轻量化的推理路径。

三、总体技术框架

本工作将全部优化分为系统层与算法层。下图给出最终推理路径，并用三种颜色区分：**白底**为模型本身的原始结构（结构未改动）、**蓝底**为系统层优化、**橙底**为算法层优化；实线为数据流，虚线表示某项优化附着在相邻的原始计算步骤上。





数据流自上而下：图像经自适应分辨率、图块嵌入进入 ViT 编码器，再经视觉 token 剪枝、Patch Merger，与分词后的问题文本拼接进入 LM 的 prefill；prefill 产出首 token 后进入 decode 循环——预测 n-gram 候选 → 一次前向验证 → 接受 1~5 个 token → 在线停止判断，循环往复直至停止后返回回答。图中术语将于第四、五章逐一展开。

四、初赛：系统级优化

初赛在不改动算法、不降低精度、不降低分辨率的前提下，仅以系统工程将 TTFT 由约 80 ms 压缩至约 20 ms、吞吐由约 55 tok/s 提升至约 299 tok/s。

4.1 注意力与 KV 缓存

问题：

1. **注意力掩码开销**：原生注意力实现需显式构造完整的注意力掩码矩阵（标记每个 token 能关注哪些位置），其大小随序列长度按平方增长，既占显存又拖慢计算。
2. **decode 重复计算**：decode 阶段每一步都重新计算当前 token 与全部历史 token 的注意力，存在大量重复计算。
3. **KV 缓存碎片**：框架默认的动态 KV 缓存（DynamicCache）随 decode 逐步追加，反复申请新内存、产生碎片。
4. **内存布局不匹配**：DynamicCache 的内存布局与高效注意力内核所要求的不一致，导致每生成一个 token 都要把整块缓存（约 235 MB）重排并复制一遍。

方案：

1. **prefill 免掩码**：prefill 阶段改用 Flash Attention 的变长序列接口（flash_attn_varlen），以序列边界描述代替完整掩码。
2. **decode 免重算历史**：decode 阶段改用 Flash Attention 的 KV 缓存接口（flash_attn_with_kvcache），直接读取历史 KV，不再重算历史注意力。

- 3. **StaticCache 预分配**：改用 StaticCache（静态 KV 缓存），在推理开始时一次性预分配固定缓存，消除动态追加与碎片。
- 4. **布局对齐 + 原地更新**：预分配时即按注意力内核要求对齐内存布局，消除每步的重排拷贝；记录有效长度的计数器也改为在原内存上直接更新，省去每步重新分配。

下表对比原生实现与本方案：

环节	原生实现	本方案
注意力掩码	显式构造完整掩码矩阵，访存随长度平方增长	Flash Attention，免显式掩码，访存随长度线性增长
decode 历史注意力	每步重算与全部历史的注意力	直接读取 KV 缓存，不重算历史
KV 缓存分配	随 decode 动态追加，产生碎片	推理开始时一次性预分配固定缓存
缓存内存布局	与注意力内核不匹配，每步整块重排（约 235 MB）	预先对齐内核要求，无需重排

效果：prefill 阶段的注意力耗时下降约四成，decode 阶段每步的注意力耗时下降约一半，每步缓存拷贝量由约 235 MB 降至可忽略。此项改造完成后，TTFT 由约 80 ms 降至约 61 ms，吞吐由约 55 tok/s 提升至约 159 tok/s。

4.2 算子融合与专用 CUDA C++ 内核

问题：

- 1. **碎片化小内核**：框架把一步前向拆成数十个细小 GPU 运算（归一化、RoPE 旋转位置编码、激活、加法等），在批大小为 1、每步计算量很小时，内核启动开销与中间结果反复进出显存成为主要瓶颈。
- 2. **GEMV 利用率低**：批大小为 1、每步只产一个 token 时，矩阵乘退化为矩阵向量乘（GEMV），cuBLAS 等通用矩阵库对这种形状的硬件利用率很低。
- 3. **线程空转**：部分通用内核因矩阵维度不是 2 的整数次幂，约四分之一的计算单元闲置。
- 4. **ViT 图块嵌入用 Conv3d**：ViT 图块嵌入用 Conv3d 实现，存在可压缩的算子开销。
- 5. **gate/up 两次矩阵乘**：MLP 的 gate、up 是两个独立投影，需两次矩阵乘、并两次读取同一份输入。

方案：

- 1. **Triton 融合内核**：把注意力前处理（QKV 拆分 + RMSNorm 归一化 + RoPE）合并为一个、层内"残差相加 + 归一化"合并为一个；并利用 Pre-Norm 结构（每层在计算前先做一次归一化）把下

一层入口的归一化前移到上一层结尾，成批消除数百次内核启动。

- 2. **CUDA C++ GEMV 内核**：为 decode 最频繁的矩阵向量乘重写专用内核，把显存带宽利用率由不足一成提升至接近满载。
- 3. **CUDA C++ 激活内核**：门控激活（SwiGLU，即 SiLU 激活后逐元素相乘）也用 CUDA C++ 按实际维度精确分配线程，消除空转。
- 4. **ViT 图块嵌入算子替换**：ViT 图块嵌入原本用 Conv3d，但由于其卷积核大小与步长都等于一个图块、各图块互不重叠，它在数学上等价于"把每个图块拉平后做一次线性投影"；因此改用更轻量、对计算图更友好的 F.linear 实现，结果完全相同，却省去了卷积算子的额外开销（im2col、专用卷积内核调度等）。
- 5. **gate/up 投影合并**：把 MLP 的 gate、up 两个投影权重沿输出维拼接，使上行投影由两次矩阵乘合并为一次，减少一次内核调用与一次对输入的重复读取。

主要融合项及其覆盖范围如下（"融合前的算子"列给出被合并的原始操作）：

融合项	融合前的算子（原始操作）	融合前内核数	融合后
LM 注意力前处理	QKV 拆分 → 对 Q、K 各做一次 RMSNorm → 对 Q、K 各做一次 RoPE 旋转 → 多次 transpose	约 18	1
ViT 注意力前处理	QKV 拆分（unbind）→ 类型转换 → 对 Q、K 各做一次 RoPE → 多次 contiguous	约 8	1
层内残差相加 + 归一化	残差相加 → 归一化 （RMSNorm/LayerNorm，其本身又展开为 pow / 求均值 / rsqrt / 相乘等若干步）	7~8	1
跨层入口归一化前移	每个中间层入口原本各自单独做一次归一化（LM 前 27 层 + ViT 前 23 层）； 融合后这些归一化被上一层末尾的"残差 + 归一化"内核顺带算掉，不再单独发起	约 250	0
SwiGLU 门控激活	SiLU 激活（作用于 gate）→ 与 up 逐元素相乘	2	1
gate/up 投影合并（权重拼接）	gate、up 两次独立矩阵乘	2	1
首/末层独立归一化	无法跨层吸收的归一化（首层入口、模型末层的 RMSNorm/LayerNorm），PyTorch 原生展开为 pow → 求均值 → 加 eps → rsqrt → 相乘 → 类型转换约 6 步	每处约 6	1

效果：算子融合压缩了 decode 阶段每步的耗时与启动开销。融合后吞吐由约 159 tok/s 提升至约 218 tok/s，TTFT 降至约 51 ms；叠加 CUDA C++ 专用内核后，decode 阶段每步耗时降至约 3.3 ms，吞吐提升至约 299 tok/s，decode 阶段显存带宽利用率达约 98%。

4.3 计算流程固化

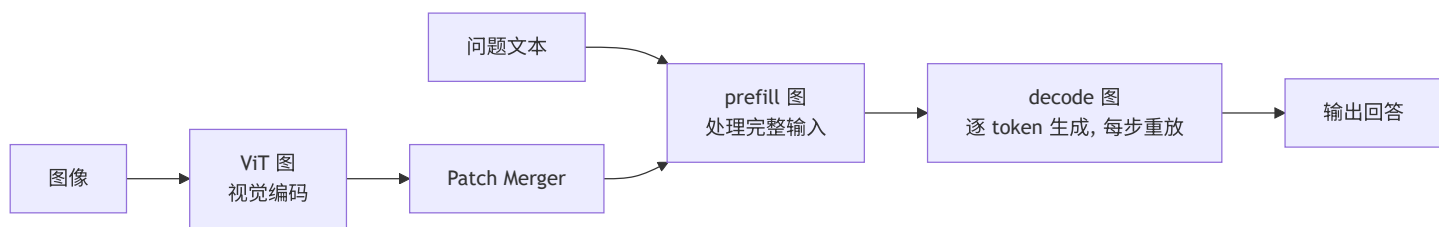
问题：

1. **调度开销：**即便完成算子融合，墙钟时间仍远大于 GPU 计算时间——ViT 24 层及 LM 每层都有数十个内核要逐个从 Python 端发起，且 decode 每步还要走一轮 Python 循环。
2. **动态形状阻碍图捕获：**各阶段都有随输入变化的动态行为（decode 的 token 回写、prefill 的变长输入、ViT 的动态最大序列长度），直接录制静态图会失败。
3. **prefill 末位冗余：**prefill 会对全部输入位置都算最终归一化与 lm_head 投影，但只有最后一个位置的输出用于产生首 token，其余均为冗余计算。

方案：

1. **三重 CUDA Graph：**把视觉编码、prefill、单步 decode 分别预录制为静态图、执行时一次性重放，把“逐内核调度”降为“每图一次重放”。
2. **消除动态因素以固定形状：**
 - **decode：**把 token 回写、位置递增等步骤一并固化入图。
 - **ViT：**把动态的最大序列长度改为预设常量；LM 的位置编码也由原生 3D M-RoPE（其构造含 CPU-GPU 同步）改为扁平的顺序位置。
 - **DeepStack：**暂时关闭（其作用是补充细粒度视觉细节，经逐样本验证关闭后对核心答案影响可忽略）。
 - **prefill 桶 + padding：**预设少数固定长度作“桶”（192、256、434、704、800、1056，主档 434；ViT 另按视觉 token 数分桶）；对真实长度 L 的输入选 $\geq L$ 的最小桶，真实序列放前部、其余作 padding，用 Flash Attention 的 `cu_seqlens=[0, L]` 告知真实边界——填充位不被关注、不影响结果（连清零都省），故每个桶只录一张图即可覆盖全部样本。
3. **prefill 只算末位输出：**prefill 仍会把全部输入位置过完 28 层、并写好各位置的 KV 缓存；只是在最后一步“最终归一化 + lm_head 投影到词表”时，只对最后一个位置的隐藏状态做计算（其 logits 用于选出首 token），跳过其余位置的隐藏状态。这里跳过的只是这些位置在末步的投影计算，**token 本身及其 KV 缓存都完整保留**——因为只有最后一个位置的 logits 用于产生首 token，其余位置再做词表投影纯属冗余。

三个阶段被分别固化为静态图后，整条推理路径如下（图与图之间由少量桥接步骤串联）：



效果：三重 CUDA Graph 是降低 TTFT 的主力，它与下面 4.4、4.5 两项一起把 TTFT 由约 51 ms 降至约 20 ms。最终 TTFT 的构成如下，约九成已是纯 GPU 计算时间，框架调度开销基本被消除：

阶段	耗时	占比
ViT 图重放	约 7.7 ms	约 39%
prefill 图重放	约 9.8 ms	约 50%
其余（patch embed、Merger、首 token 投影等）	约 2.5 ms	约 11%

4.4 ViT 位置编码缓存

问题：

- 1. **位置编码重算与同步：**ViT 支持动态分辨率，每张图都要重算加性位置嵌入、RoPE、cos/sin、cu_seqlens 等位置相关量，其中含频繁的 CPU-GPU 同步，单图约 1.85 ms。

方案：

- 1. **按网格形状缓存：**上述各量只取决于图片的网格形状（grid_thw，即 patch 网格的高×宽），与图片内容无关。又因为 max_pixels 上限会把较大的图等比缩小、并对齐到 32 的倍数，所有图片最终只落在少数几种网格尺寸上，很多样本因此共享相同的 grid_thw。据此按网格形状做字典缓存：首次算出后存下、后续遇到相同形状直接复用。本数据集约三分之二样本共享相同网格形状（704、768 两类视觉 token 数占主导），命中率很高，可跳过重复计算与 CPU-GPU 同步。

效果：位置编码缓存使 TTFT 进一步下降约 3 ms。

4.5 绕过 nn.Module 调用开销

问题：

- 1. **无效的 hook 检查：**PyTorch 每次层调用都会先走一遍 nn.Module 的 hook 检查链（为训练期钩子、profiling 等服务），ViT 与 LM 合计约 350+ 次调用，在纯推理下这些检查纯属无效开销。

方案：

1. **函数式直调**：把推理路径上的简单线性层改用函数式 `F.linear` 调用、复杂模块直接调用 `.forward()`，跳过 hook 检查链、直达核心计算。

效果：该优化使 ViT 阶段约下降 4 ms。

五、复赛：算法级优化

复赛在初赛系统层之上叠加算法优化。

5.1 权重 INT8 量化

问题：

1. **带宽受限**：decode 阶段每生成一个 token 都要把约 3.49 GB 权重读入一次；初赛已将带宽利用率提升至接近满载，进一步提速只能减少需读取的权重字节数。
2. **量化误差风险**：直接量化会引入数值误差，尤其输出投影（LM Head）的误差会直接影响最终选词。

方案：

1. **Weight-only INT8 (W8A16) + 逐通道量化** (W8A16 中 W=权重、A=激活，即权重存为 8 位整数、激活仍用 16 位浮点)：
 - **做法**：将语言解码器每层的线性权重（QKV、O、gate/up、down）与 lm_head 量化为 INT8（该量化在模型加载时一次性完成，连同每通道 scale 一起存于显存，不增加每步推理时延）；decode 与验证时由 GEMV 内核直接读取 INT8、在内核内用 scale 边读边还原为 FP16 参与计算（每步只读 INT8 半量字节，即省下带宽）。
 - **逐通道 (per-channel)**：为权重矩阵的每个输出通道（即每一行）单独配一个缩放系数（scale，用于把整数还原为浮点），比整个矩阵共用一个 scale 更贴合各通道的数值范围、把量化误差压得更小。
 - **范围与收益**：上述 INT8 权重用于 decode 与推测验证路径，使每步权重读取量由约 3.49 GB 降至约 1.75 GB；ViT 与大部分 prefill 仍用 FP16（被量化的权重另存一份 FP16 副本供 prefill 使用）。
 - **prefill 后段更换为 W8A8**：prefill 的 LM 后段第 18–27 层 gate/up 投影进一步走 INT8 W8A8（权重与激活均 INT8，经 DeepGEMM/SmoothQuant 实现），降低 prefill 后段的矩阵乘开销。
2. **LM Head 两段式选词**：先用 INT8 粗算全词表分数、选出分数最高的 8 个候选（Top-8），再仅对这 8 个候选用 FP16 精确权重重算并定下最终 token，兼顾速度与精度。



效果：INT8 量化将 decode 阶段的权重读取量减半，在准确率基本不变的前提下提升吞吐；输出投影的两段式选词进一步避免了量化引入的选词偏移。该方案亦为后续推测解码提供了更低成本的单步计算。

5.2 推测解码

问题：

1. **串行瓶颈：**即便单步计算已逼近带宽极限，自回归每次前向仅产生一个 token 的串行性仍构成吞吐的上限。

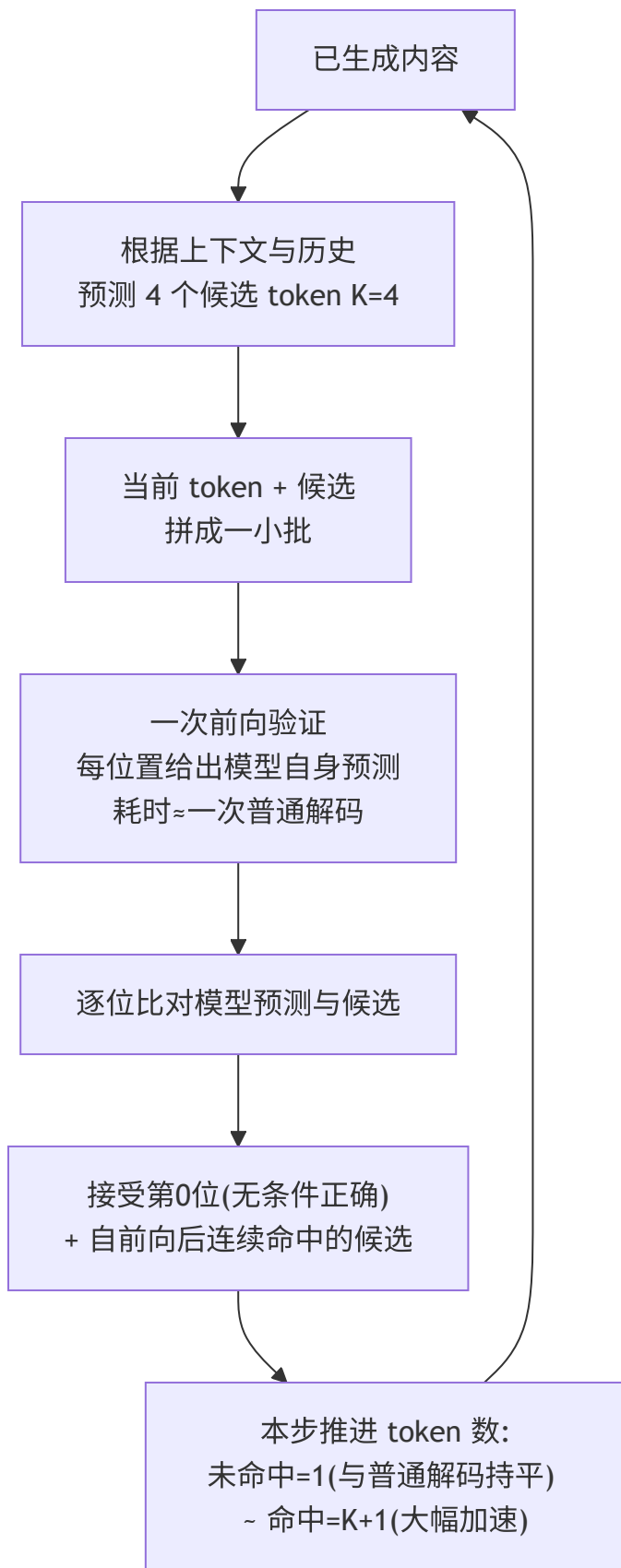
方案：采用基于 n-gram 的推测解码，无需额外的草稿模型。其流程分为"预测候选"与"一次验证"两步。

(1) **预测候选：**本方案一次预测 $K=4$ 个候选 token，候选来自 n-gram 统计。预测单个候选时，以当前结尾最多 3 个词作为上下文（连同要预测的那个词即构成一个 4-gram），在已生成文本与问题中回溯查找该上下文之后出现频率最高的词（而非最近一次出现的词）作为候选；若 3 个词的上下文无匹配，则回退到 2 个词（3-gram）、1 个词（bigram）继续查找。每预测出一个候选，就把它接到上下文末尾再预测下一个，依次得到候选序列 c_1, c_2, c_3, c_4 。除当前样本的上下文外，候选来源还包含一份**跨样本 n-gram 记忆**：把先前样本生成过答案 n-gram 持续累积下来供后续样本查询，进一步提高候选命中率。

(2) **一次验证：**将"当前 token + K 个候选"共 $K+1$ 个 token 拼为一个小批量，送入模型做一次前向。该前向会在每个位置上同时给出模型自身的下一 token 预测：第 0 位给出"当前 token 之后真正应生成的 token"，第 1 位给出"若 c_1 成立其后应生成的 token"，依此类推。验证即将模型在各位置的预测与候选逐位比对：第 0 位的预测是模型的真实输出，必然接受；其后若 c_1 与第 0 位预测一致，则 c_1 正确、第 1 位的预测随之可信并继续接受，如此自前向后连续接受，直至遇到首个候选与模型预测不一致处停止。

该机制的收益是非对称的：命中时大幅加速，未命中时也不低于普通解码（最坏情况下与普通解码持平）。具体而言：

- 无论命中与否，本次验证都至少产出第 0 位那个必然正确的 token，它恰是普通逐 token 解码本应生成的那个。因此即便全部候选预测错误，本步仍推进 1 个 token，与普通解码完全一致，没有损失。
- 由于 decode 阶段为显存带宽受限、单步耗时几乎只取决于权重读取量，而一次验证（处理 $K+1$ 个 token）所读取的权重与普通解码处理 1 个 token 几乎相同，因此一次验证的耗时约等于一次普通解码。一旦候选命中，本步即以一步的代价一次推进 2 至 $K+1$ 个 token，收益明显。
- 在严格验证下，被接受的 token 均为模型自身的真实预测，与逐 token 贪婪解码一致；后续的低精度与跳层近似仅在答案主体稳定后启用，并通过离线准确率回归控制风险（见 5.6）。



效果：推测解码将"单次前向仅产一个 token"转变为"单次前向可产多个 token"。本方案 K=4，一次验证最多推进 5 个 token（接受 1 至 5 个），在严格验证保持答案与模型预测一致的前提下提升生成吞吐，是复赛吞吐提升的主要来源之一。

5.3 自适应分辨率与视觉 token 精简

问题：

1. **prefill 开销过高**：视觉 token 的数量直接决定视觉编码与 prefill 的开销，减少视觉 token 可同时降低 TTFT、提升吞吐。
2. **减少 token 数量同时保证准确率**：本数据集大量题目依赖图中的细小文字、数字、品牌等信息，分辨率一旦降低就会丢失关键证据、损害准确率。

方案：

1. **自适应分辨率**：按问题类型选择分辨率——对依赖细节的“文本敏感”题（如“图中的价格是多少”“车牌号是什么”“书的标题是什么”）保持高分辨率（像素上限 401408，约合 634×634 ）以护准确率，对不依赖细节的题（如“这是什么颜色”“图中有几个人”）采用较低分辨率（像素上限 262144，约合 512×512 ）以降低开销，依问题中的关键词判定。
2. **ViT 后 PruMerge 精简**：对视觉编码后、送入语言模型前的视觉特征做 PruMerge 精简——以每个视觉特征向量的范数（可粗略理解为该特征被激活的强弱）衡量重要性，保留最重要的约一半（保留比例 0.5、至少 64 个视觉 token）；该精简置于 ViT 之后，以免破坏已固化的视觉计算流程。

效果：自适应分辨率在不牺牲文字题准确率的前提下，为不依赖细节的题目降低了部分视觉编码与 prefill 开销，带来 TTFT 与吞吐的稳定收益。

5.4 在线停止策略

问题：

1. **何时停**：需在模型已给出有效答案、开始产生无意义结尾时及时停止。
2. **凭什么停**：停止判断只能依据生成过程本身可解释的信号，不得借助任何离线信息。

方案：

1. **在线停止三规则**：均在生成下一个 token 之前判断、命中即停（使无意义内容自始不产生而非事后删改）：
 - (a) 已构成简短完整的答案（如名词短语或数值）并以句号结尾；
 - (b) 连续重复 16 个相同 token（如“the the the...”）；
 - (c) 出现半截引导语后开始反复（如“The image shows”后陷入重复）。
2. **直接作答 + 模板屏蔽**：在生成第一个 token 时降低“Based on”“According to”等套语开头词的概率（direct-answer-start）以直接作答，并在生成与候选筛选阶段屏蔽 assistant、user、system 等对话模板 token，避免模板串入答案。

效果：在线停止使回答长度自然、结尾干净，并与推测解码相配合，在保证答案可读的同时维持高吞吐。

5.5 后段 KV 注意力窗口

问题：

1. **长序列注意力开销：**decode 每一步都要让当前 token 与"全部历史 KV"做注意力，而这段历史 = 图像视觉 token + 文本 prompt + 已生成 token。对 VLM 而言，仅图像与 prompt 就常达数百至上千个 token，是注意力开销的大头（与已生成多少 token 关系不大）。

方案：

1. **后段 KV 窗口：**用 Flash Attention 的滑动窗口把每步注意力限制在"最近若干个 KV"上，而非回看全部历史。
 - **窗口大小：**常规阶段为最近 484 个 token，生成超过 24 个 token 后进一步收紧到最近 128 个。
 - **128含义：**这里的 128 指完整 KV 序列（图像 + prompt + 已生成）中最近的 128 个位置，与"吞吐测试最多生成 128 个 token"无关——prompt 本身就有数百个 token，把注意力范围从上千个收紧到最近 128 个，能显著降低每步注意力的读取与计算量。
 - **影响与范围：**对答案较短、关键信息集中在邻近上下文的 VQA 任务，较早历史对后续 token 影响很小，收紧窗口对答案基本无影响；该窗口同时作用于单步 decode 与推测验证两条路径，并各自预录制对应的固定形状计算图。

效果：后段 KV 窗口把每步注意力涉及的 KV 由"随全长增长"降到固定约 128 个，在长输出阶段进一步提升吞吐，而对答案质量的影响经核验可忽略。

5.6 算法与系统的协同（关键工程化）

问题：

1. **框架开销会吞掉算法收益：**复赛引入的量化、推测解码等算法，若仅照搬论文或开源实现直接接入，往往会带来额外的 CPU-GPU 同步、随输入变化的张量形状、临时内存申请等开销。在批大小为 1、每步计算量本就很小的场景下，此类框架开销会抵消算法本身的收益，甚至导致整体变慢。也就是说，一项算法能否带来加速，不仅取决于其在论文中的理论收益，更取决于其在本系统中能否被高效地实现。

方案：本工作将每一项算法优化重新实现为"对计算图友好、几乎不增加调度开销"的形态：

- **为验证步骤定制融合内核：**推测解码的验证需对多个候选 token 一次性前向。普通 decode 路径里，门控 MLP 的"矩阵乘"与"SwiGLU 激活"是两个分开的内核；本方案为验证步骤专门写了一个内

核，把这两步合并为单个 INT8 内核完成，使一次验证仅发起少量内核调用。

- **验证路径的跳层与跳投影近似：**验证只需判断候选能否被接受、不依赖验证步自身的输出，故可在该路径跳过部分计算。
 - **跳层：**整层跳过第 13–15 层、跳过第 20/21/23–26 层的 MLP、最后一层写入 KV 后即返回。
 - **盲接受：**在答案主体稳定后，对其后的低风险延续 token 采用近似验证（不再用 lm_head 核对、直接采用 n-gram 候选）；答案主体区仍走完整 lm_head 核对，且该近似经离线验证确认不影响 VQA 软匹配准确率。
 - **边界：**跳过的只是出词投影，被接受 token 的 K/V 仍照常写入缓存；这些近似只作用于验证路径，prefill 与单步 decode 仍走完整计算，对答案影响经核验可忽略。
- **多档计算图覆盖变长输入：**为单步 decode（固定 1 个 token）、推测验证（固定 K+1=5 个 token）、以及不同长度的 prefill（桶 192、256、434、704、800、1056，主档 434）分别预录制固定形状的计算图，运行时按实际长度选最接近的一档，既覆盖了视觉 token 数剧烈变化带来的变长输入，又保证每次仅需一次图重放。
- **相同图像前缀的缓存复用：**同一图像被多次询问时，其视觉前缀的中间结果仅需计算一次并缓存，后续仅补算新增的问题部分，从而避免重复的视觉前缀计算。
- **相同图像的视觉编码缓存：**以图像内容的哈希为键，缓存视觉编码（ViT + Patch Merger）输出的视觉 embedding（LRU，容量 64）；同一图像再次出现时直接复用，跳过整个视觉编码。

效果：上述工程化改造使算法优化真正落地为端到端加速，而非停留于"理论上更快"。依靠算法与系统的协同，复赛在系统层已接近极限的基础上，将吞吐由数百 tok/s 提升至数千 tok/s。

六、所有优化方向总结

阶段	优化	主要作用
初赛	注意力与 KV 缓存改造（Flash Attention + StaticCache 布局对齐）	降低 TTFT、提升吞吐
初赛	算子融合（Triton 跨层融合 + CUDA C++ GEMV/激活内核）	降低 TTFT、提升吞吐
初赛	三重 + 多档 CUDA Graph 计算图固化	降低 TTFT、提升吞吐
初赛	ViT 位置编码缓存	降低 TTFT
初赛	绕过 nn.Module 调用开销	降低 TTFT
复赛	权重 INT8 量化（W8A16）+ lm_head 两段式选词	提升吞吐、保准确率
复赛	n-gram 推测解码	提升吞吐

阶段	优化	主要作用
复赛	自适应分辨率 + 视觉 token 精简	降低 TTFT、提升吞吐
复赛	在线停止 + 直接作答 + 角色 token 屏蔽	提升吞吐、 保证答案干净
复赛	后段 KV 注意力窗口	提升吞吐
复赛	验证专用融合内核 + 跳层/跳投影（盲接受）	提升吞吐
复赛	相同图像前缀的 KV 缓存复用	降低 TTFT

提交包文件与实现落点（提交包共 10 个文件，报告所述优化均可在其中找到对应实现）：

文件	实现落点
evaluation_wrapper.py	主推理路径（vLMModel）：ViT/prefill/decode 流程、三重与多档 CUDA Graph、推测解码（lookahead）、各类缓存复用、自适应分辨率与视觉剪枝、在线停止等
fused_ops.py	Triton 融合算子（QKV+QK-Norm+RoPE、残差+归一化、SwiGLU、首/末层归一化等）
cuda_gemv.py	自定义 CUDA 扩展的加载器（预编译 .so → 即时编译 → PyTorch 三级回退）
cuda_gemv_ext/gemv_kernel.cu	核心 CUDA 内核：INT8/FP16 GEMV、SiLU×Mul、KV 写入、验证路径融合内核、n-gram 相关 GPU 辅助
cuda_gemv_ext/cutlass_mlp.cu	CUTLASS MLP 内核（实验性，默认推理路径未启用）
cuda_gemv_ext/setup.py	CUDA 扩展编译脚本
cuda_gemv_ext.*.so	上述 CUDA 内核的预编译二进制
ngram_tables/*.pt	推测解码用的辅助 n-gram 表
README.md	提交说明

七、结论

本工作在批大小为 1、不重新训练、不离线改写模型语义权重的约束下，构建了一套系统层与算法层协同的 VLM 推理加速方案。

- 系统层（初赛）：通过注意力与缓存改造、算子融合、计算流程固化、位置编码缓存等系统工程手段，将 TTFT 由约 80 ms 降至约 20 ms、吞吐由约 55 tok/s 提升至约 299 tok/s，准确率基本无损，decode 阶段显存带宽利用率达约 98%。
- 算法层（复赛）：在系统层之上引入 INT8 量化、推测解码、自适应分辨率与在线停止等算法优化，并将每一项实现为"对计算图友好、低调度开销"的形态，使其真正转化为端到端加速；最终提交版本在 TTFT 约 20 ms 的同时将官方吞吐提升至约 5634 tok/s（约为基线的 100 倍）。